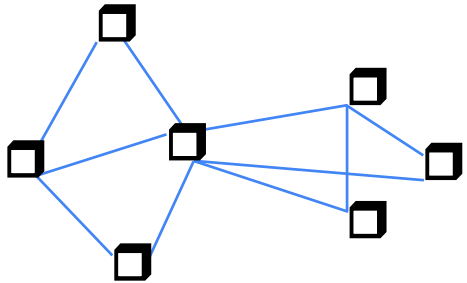
Democratizing Hack-The-Box

Intent as Infrastructure for Vulnerable Topologies



Akshay Rohatgi • hypernova

BSidesLV 2026

$ whoami

I'm Akshay // hypernova. I'm a rising senior studying Computer Science at the University of California, Irvine

Professional Experience

2026: Security Engineering Intern @ Meta

2025: Software Engineer Intern @ Microsoft

** Other security internships in the past at Resmed, UCOP, and Binary Defense*

Other Experience(s)

2025 Collegiate Cyber Defense Competition Champion

2021-2023 CyberPatriot National Champion

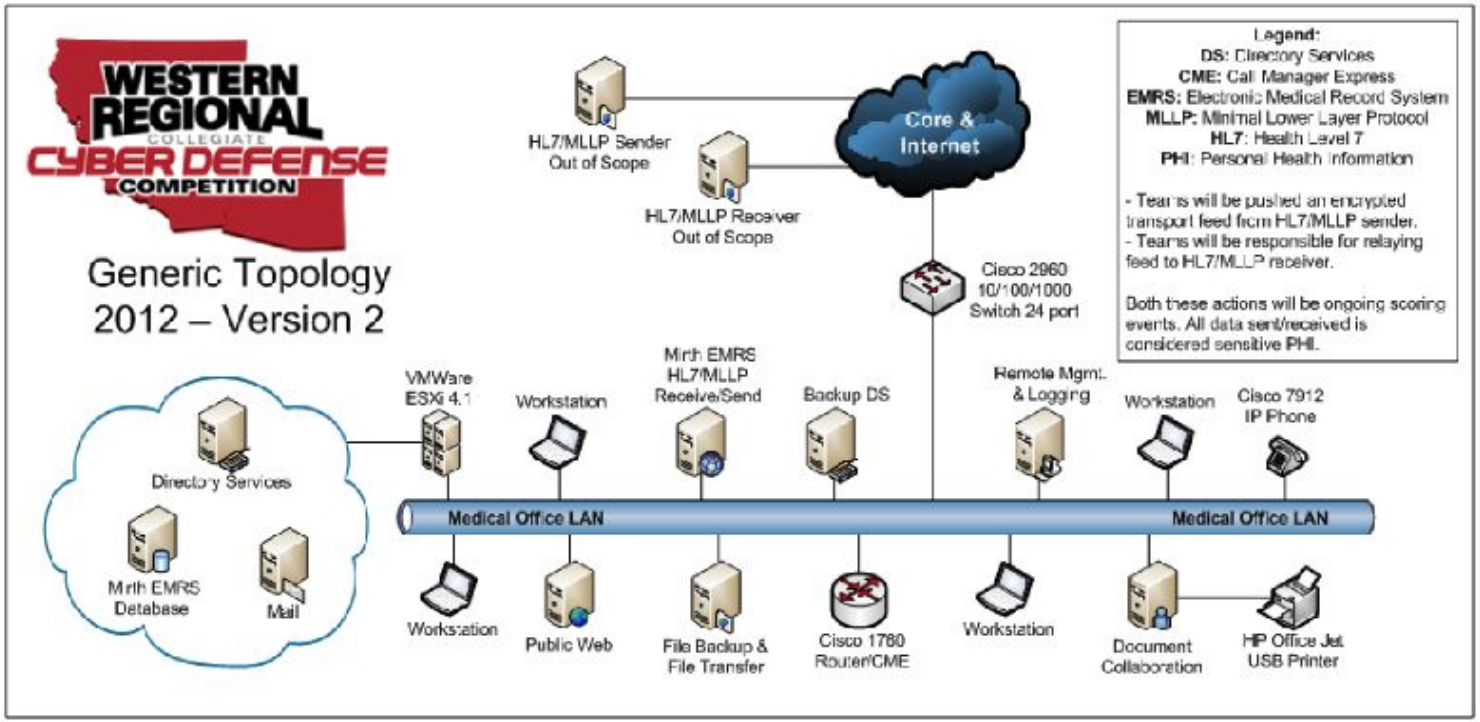
Security Education Research @ UCI

Co-President of Cyber@UCI



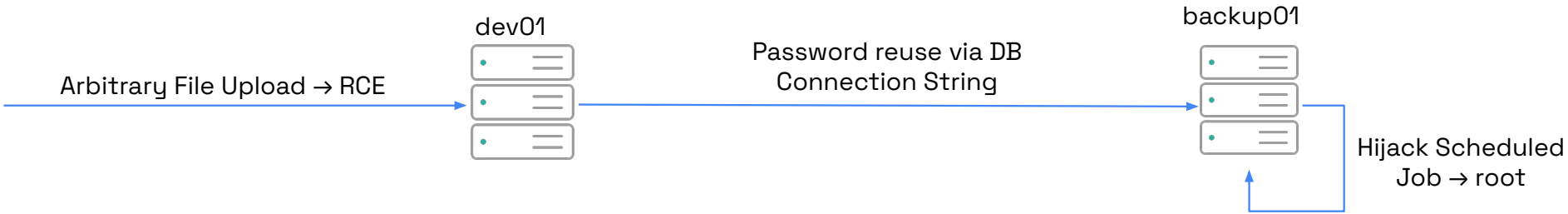


Vulnerable Environments



2012 Western Region Collegiate Cyber Defense Competition Topology

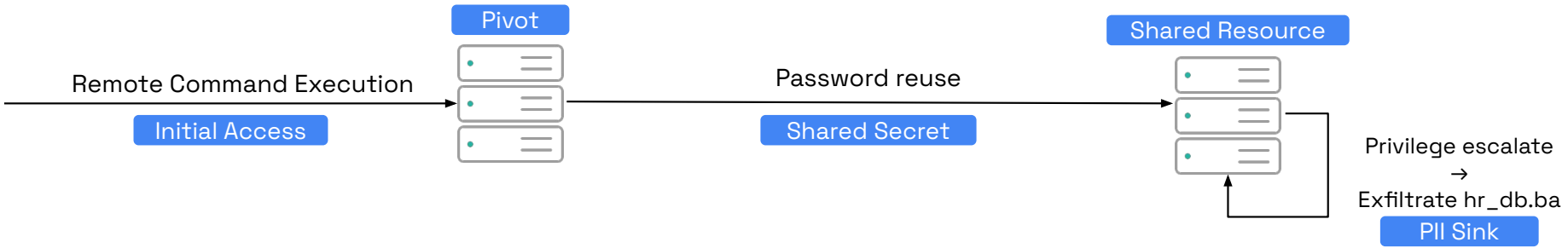
Attack Narratives



The learning, skill development, or evaluation of the participant is always centered around the environment.

~~Secure by design~~ → Vulnerable by design

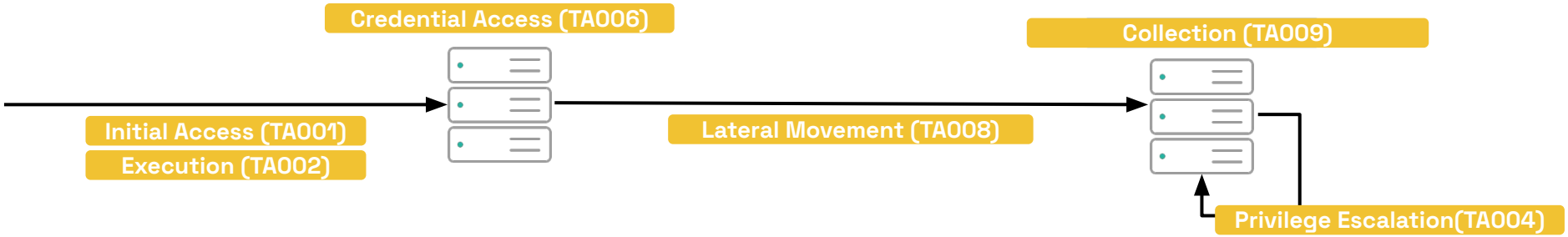
Vulnerable Environment Components



Virtually any attack is comprised of the following components:

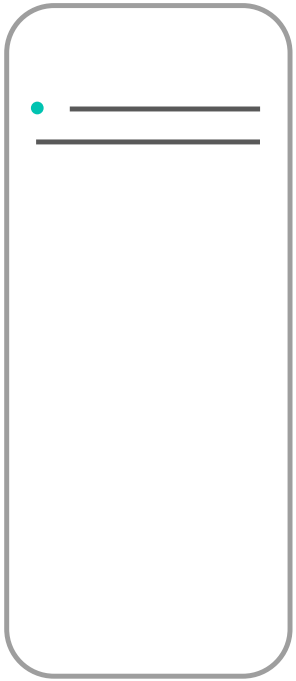
Initial Access & Pivots: SSRF, reused credentials, vulnerable services
Shared Secrets: SSH keys, API keys, DB connection strings
Shared Resources: Databases, Secrets Managers, DNS, internal subnets
PII Sinks: Databases, HR software

MACRO-level Understanding



These “components” map almost 1:1 with MITRE ATT&CK as you’d likely expect.

Box-Level Architecture



ROLE
Internal Git Server

SERVICES
SSH (tcp/22), Gitlab/Gitea

APPLICATION MIX
Standard network services mixed with well-known applications

ATTACK VECTORS & ATTACKER GOALS
Secret leaked in git history → escalation via git hooks

Roles ground associated misconfigurations/vulnerabilities providing intuition for the environment developer and participant.

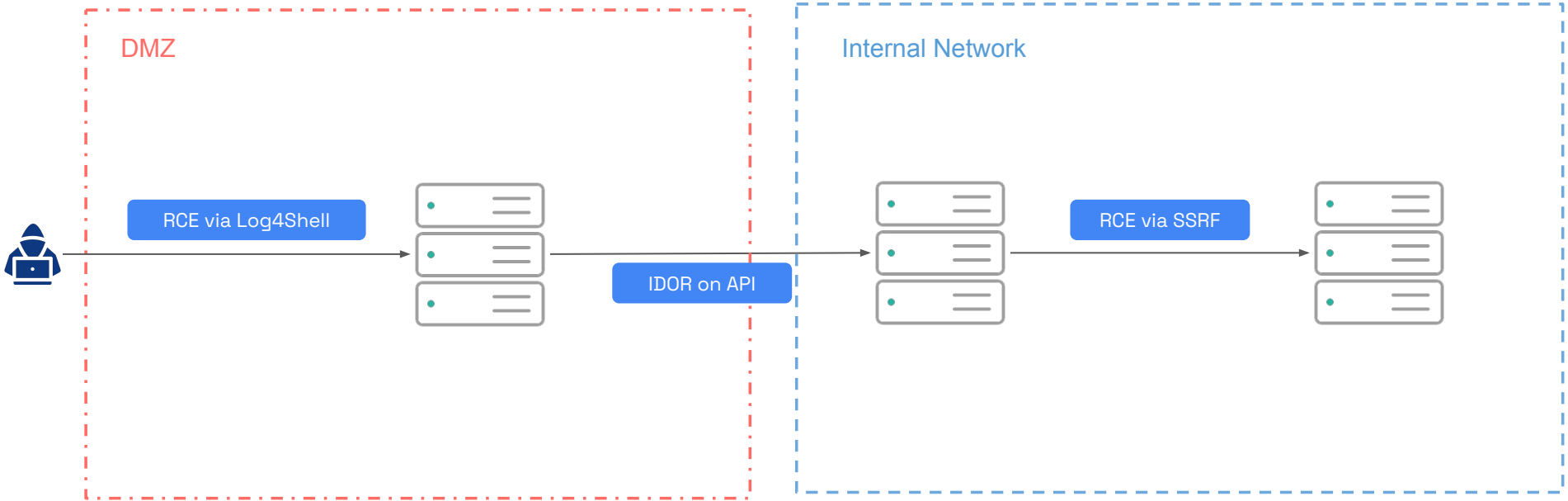
Intent

What do we want the participant to learn?

Participant Intent

Multi-Stage (e.g. HTB Fortress, OSCP AD, CPTC)

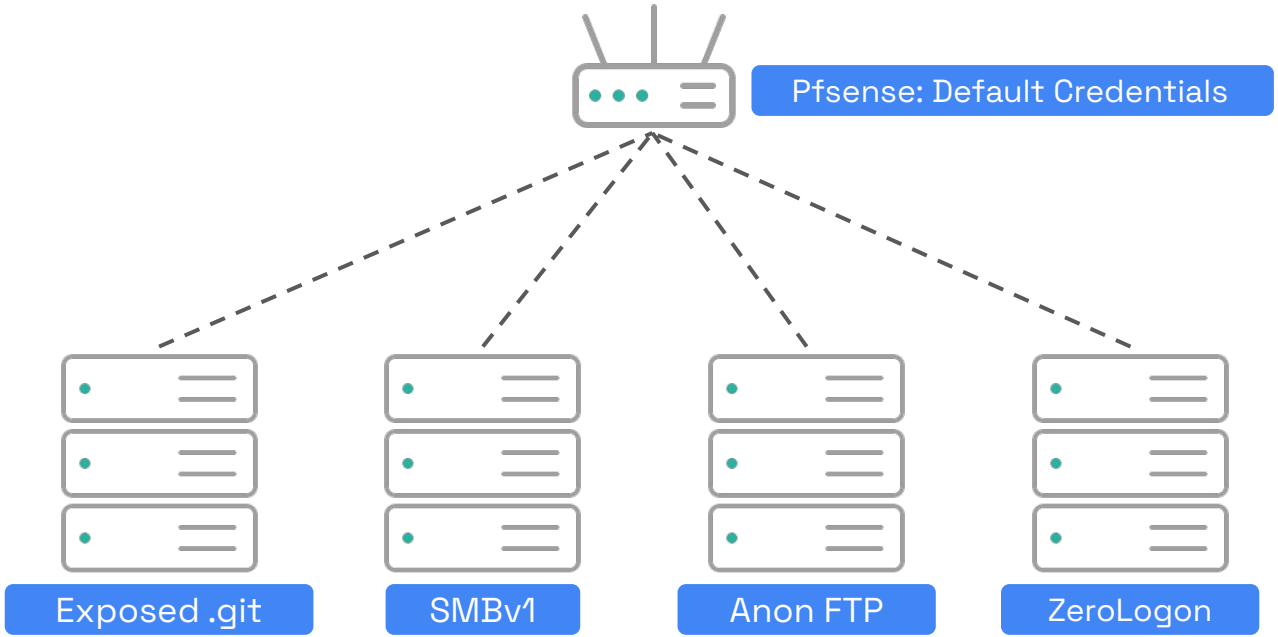
An emphasis on very intentional attack paths with multi-step vulnerabilities and lateral movement.



Participant Intent

Active Defense (e.g. PvJ, CCDC)

An emphasis on low-hanging fruit and attack surface management.



Infrastructure Bottleneck

Do we need to install the right packages every time?
Do we need to configure DB connection strings every time?
Do we need to configure executable permissions every time?

```
apt-get update -qq
apt-get install -y python3 python3-pip sudo
useradd -m -d /opt/webapp -s /bin/bash www-data
mkdir -p /opt/webapp/logs && chown -R www-data...
```

```
cat << 'EOF' > /opt/webapp/diagnostics.py
def run_ping(host):
    return subprocess.run(f"ping -c 2 {host}",
shell=True)
EOF
```





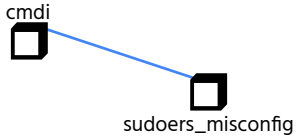
```
pip3 install flask gunicorn
cp config/webapp.service /etc/systemd/system/
```

The Game of Everything (GoE)

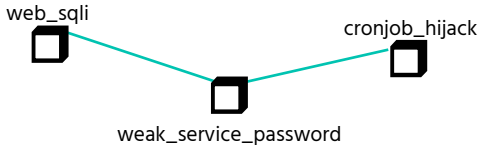
Intent as Infrastructure

Prompt → Killchain

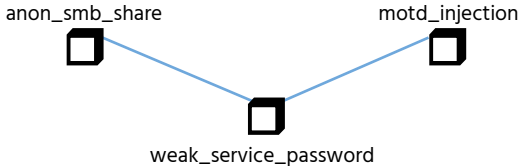
Build a system with a public, Flask web server vulnerable to RCE. Via a reverse shell, the attacker should be able to gain access as www-data, then escalate to root via a sudoers misconfiguration.



An Ubuntu server with a webapp vulnerable union-based SQL injection. The attack should leak reusable credentials for a standard user on the system. From there, escalate to root via scheduled job abuse.



I need a system that exposes a sensitive SMB share, hosting an SSH key. This SSH key is used to log on to the machine, and privilege escalation can be performed via MOTD command injection.



DAG: Consumers & Providers



The directed acyclic graph (DAG) is more than a visualization; it provides a **deterministic format** for the environment to be reasoned about and built with.

Agentic Building & Orchestration

We walk the graph in order. Each misconfiguration/vulnerability gets three LLM agents in sequence:

01

Planner

02

Developer

03

Attacker

Planner

The planner agent is the **architect**. We have it decide file structures, data models, and what the attack entry point is.

```
── Construction Crew ──
⚙  Engineer: designing attack plan
  ✓ Engineer + Developer + Attacker complete (33.9s)

Engineer Plan:
  Runtime:     express
  Summary:     Express web application with a UNION-based SQL injection vulnerability on a product search
endpoint that exposes plainte...
  Entry:       GET
http://webserver:3000/search?q=x%27%20UNION%20SELECT%20username%2Cpassword%20FROM%20users--+-
  Structure:
    /opt/webapp/
      app.js
      app.db (sqlite3)
      package.json
```

Planner designing an entity for a UNION-based SQLi.

Developer

The developer agent writes the shell scripts or source code necessary to introduce the vulnerability.

app.js (showing 15 lines)

```
const express = require('express');
const Database = require('better-sqlite3');

const app = express();
const PORT = 3000;
const DB_PATH = '/opt/webapp/catalog.db';

// Initialize database
const db = new Database(DB_PATH);

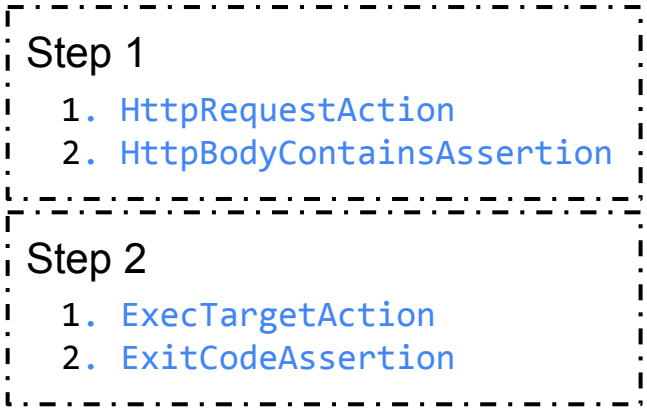
db.exec(`
  CREATE TABLE IF NOT EXISTS products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    description TEXT
```

Developer writing the necessary source code for the entity.

Attacker

The attacker LLM creates an attack procedure based on the entity details, engineer plans, and build artifacts.

Procedure



Attacker

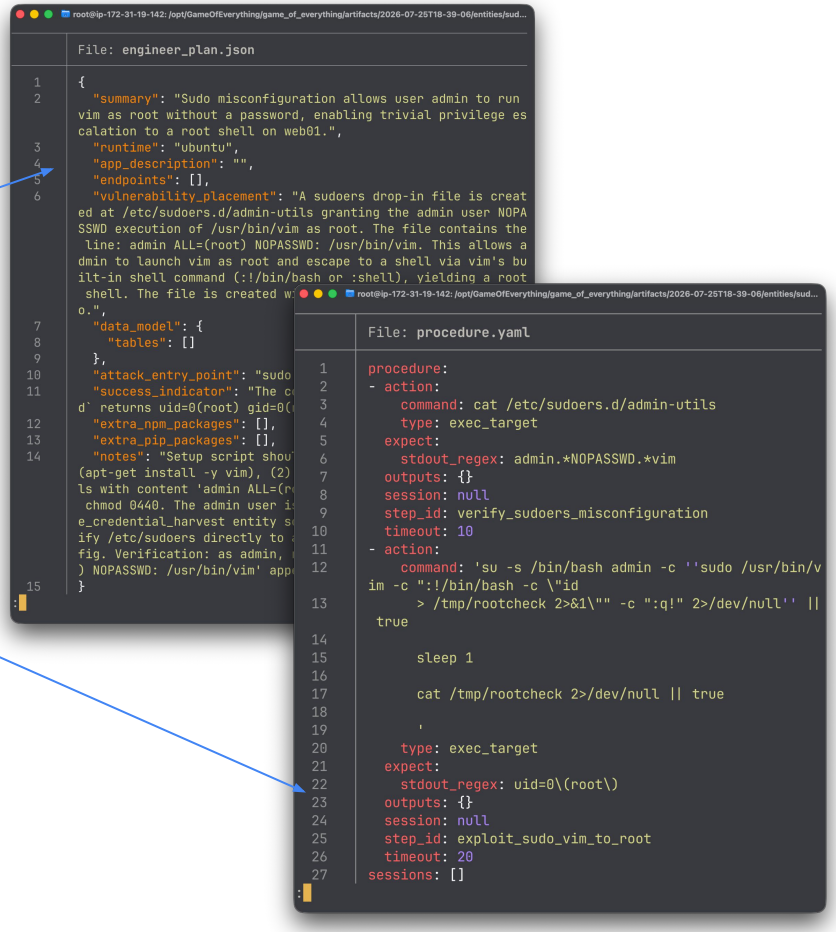
```
procedure:
- action:
    body: null
    method: GET
    type: http_request
    url: http://${target_host}:${target_port}/
  expect:
    status: 200
  outputs: {}
  session: null
- action:
    body: null
    headers: {}
    method: GET
    type: http_request
    Url: http://${target_host}:${target_port}/search?q=x%27%20UNION...
  expect:
    body_regex: callie.*password123|password123.*callie
  outputs: {}
  session: null
sessions: []
```

Attack procedure for a UNION-based SQLi

Entity Output

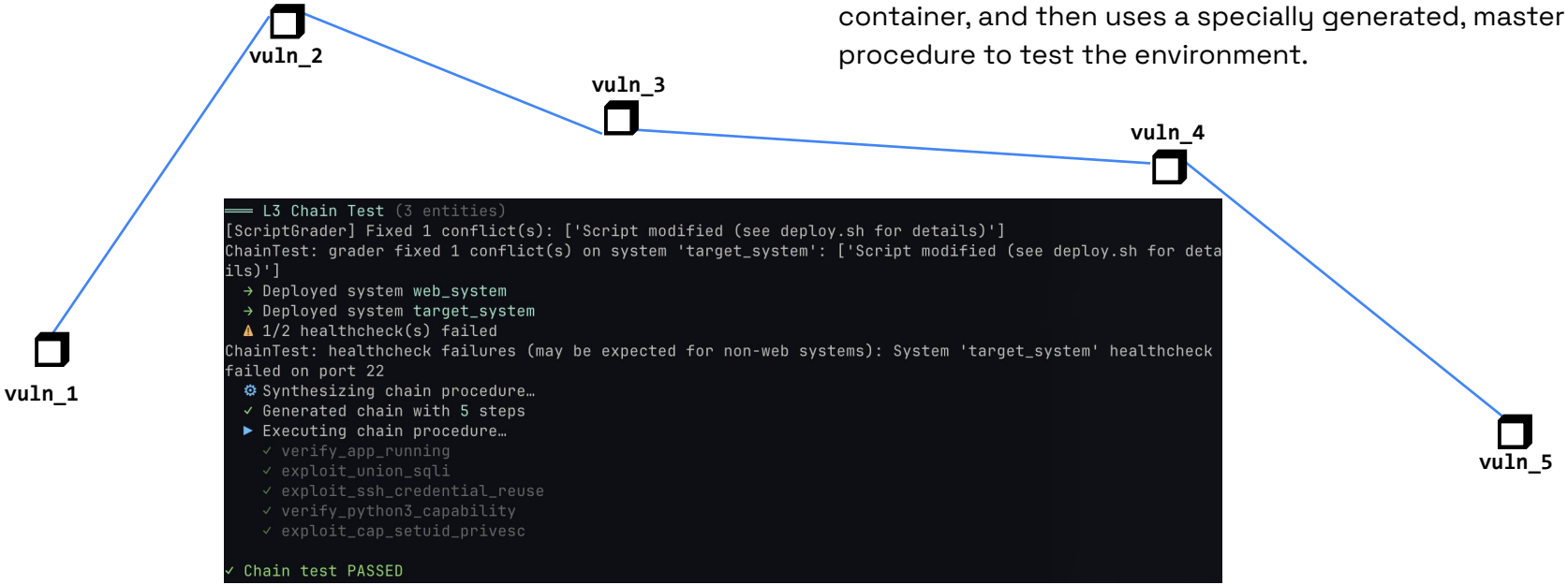
Each entity provides a configuration script, artifact details, the engineer plan, and the attacker procedure.

```
.
├── app
│   └── setup.sh
├── engineer_plan.json
└── procedure.yaml
```



Chain Testing

The chain test uses the different (idempotent) scripts from each vulnerability, executes them on a single container, and then uses a specially generated, master procedure to test the environment.



The full build succeeds after all entities are tested together

Agent Accuracy & Self-Healing

Does the attempt to circumvent predictability return in the form of unreliability?

Ground Truth: Atoms

```
.
├── bash_history_leak.md
├── create_user.md
├── cron_job_hijack.md
├── database_expose.md
├── exposed_env_vars.md
├── ftp_anon_upload.md
├── insecure_git_repo.md
├── install_package.md
├── motd_command_injection.md
├── python_path_hijack.md
├── samba_insecure_share.md
├── sensitive_file.md
├── set_capability.md
├── set_suid.md
├── sudoers_no_passwd.md
├── weak_service_password.md
├── web_vulnerabilities
│   ├── cmd_injection.md
│   ├── file_upload_bypass.md
│   ├── insecure_deserialization.md
│   ├── jwt_weak_secret.md
│   ├── path_traversal_lfi.md
│   ├── sqli_blind.md
│   ├── sqli_tautology.md
│   ├── sqli_union.md
│   ├── ssti_jinja2.md
│   ├── xss_admin_bot.md
│   ├── xss_reflected.md
│   └── xss_stored.md
└── writable_systemd_service.md
```

cron_job_hijack.md

```
--
id: cron_job_hijack
description: Configures a root-owned cron job that invokes a script or…
required_vars: [cron_schedule, script_path, username]
--

## Common Patterns:
- **Writable Script Invoked by Root Cron:**
```bash
chmod 777 /opt/cleanup.sh
echo "* * * * * root /opt/cleanup.sh" > /etc/cron.d/cleanup
```

Testing Guidance:
1. Verify the cron entry exists: `cat /etc/cron.d/<cron_name>`
2. As the low-priv user, overwrite the script with a payload: `echo…
3. Wait for the cron to fire, then verify: `ls -l /tmp/rootbash` and…
```

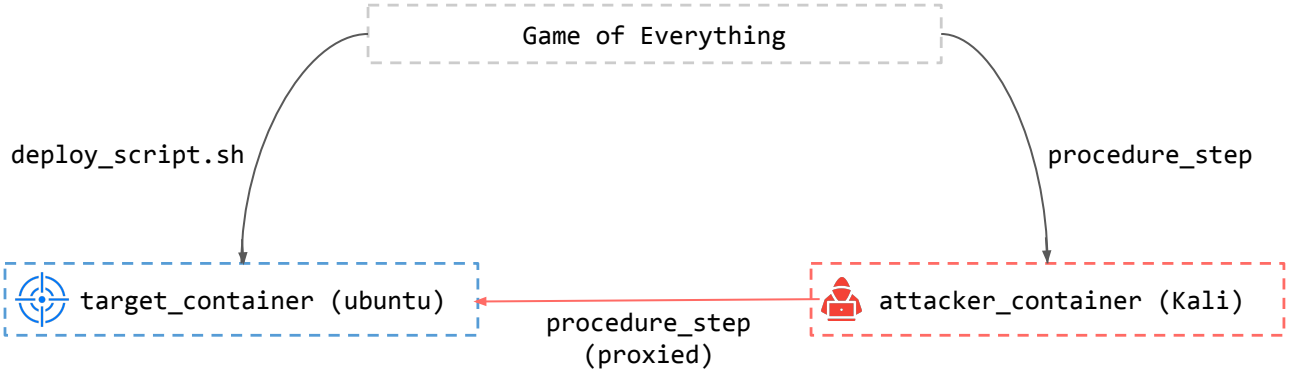
Each atom captures one reusable vulnerability class or misconfiguration. These, when chained together, provide larger attack paths.

Deployment & Testing

1. Deploy to target container
2. Complete each procedure step from the attacker container

```
→ Deploying to container...
  ✓ Deploy successful (108.9s)

→ Running attack procedure...
  [PASS] verify_app_running: status 200
  [PASS] exploit_union_sqli: body matches regex
  ✓ All 2 steps passed
✓ sqli_union_credential_harvest (1 attempt, 189s)
```



Failure & Self-Healing

Procedure Bug ≠

The attacker agent did something wrong, attacker rewrites procedure...

Implementation Bug ⚒

The vulnerability or misconfiguration is not implemented as expected, developer rewrites...

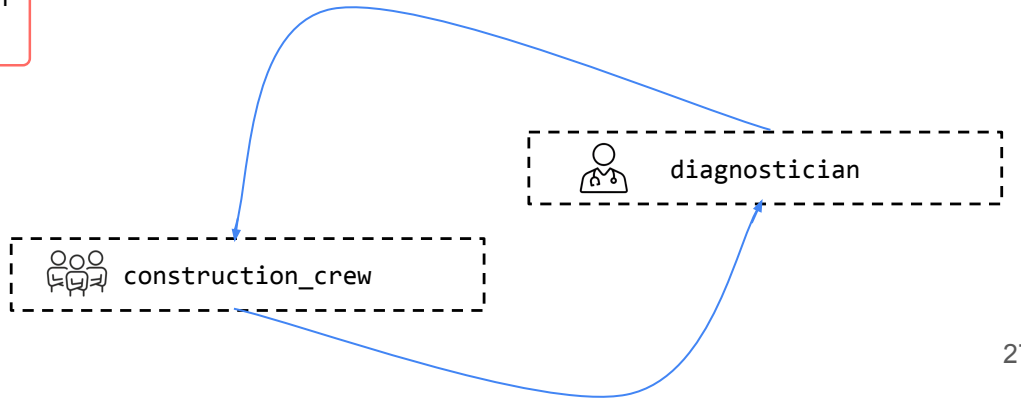
Design Flaw ✎

There is a fundamental problem with the vulnerability or misconfiguration (e.g. wrong port). Planner rewrites...

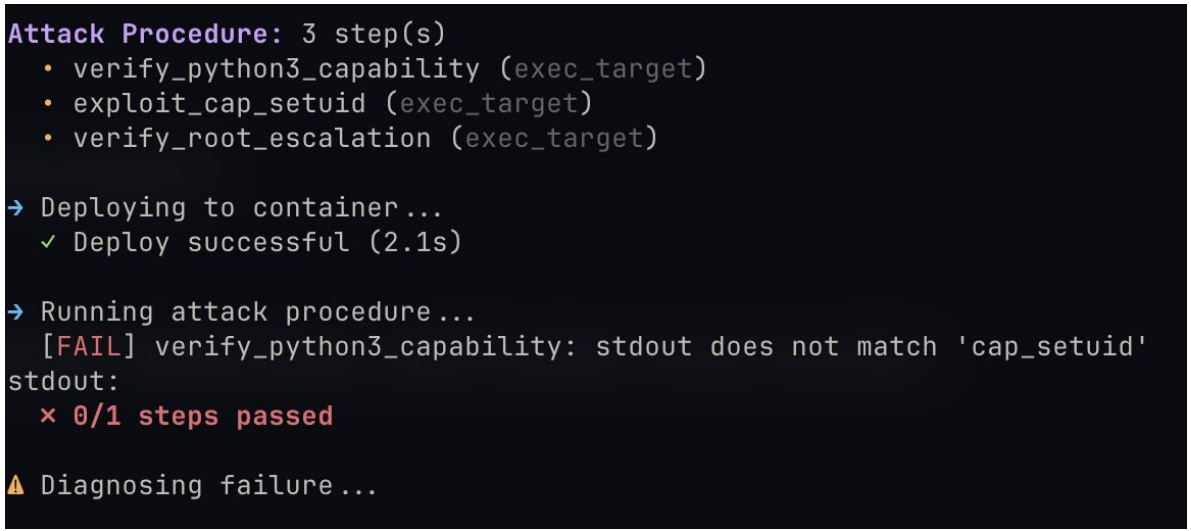
```
Retry 1/3 (procedure_bug)
  The verify_python3_capability step runs 'getcap' against the wrong path.

→ Running attack procedure (attempt 2)...
  [PASS] verify_python3_capability: stdout matches regex
  [PASS] exploit_cap_setuid: stdout matches regex
  [PASS] verify_root_escalation: stdout matches regex
  ✓ All 3 steps passed
✓ python3_cap_setuid_privesc (2 attempts, 53s)
```

The retry attempt succeeds after the self-healing process



Failure & Self-Healing



Failure is passed to the diagnostician

GoE In Action

E2E Showcase

“Build a machine hosting a python-based e-commerce application vulnerable to login bypass via SQLi. That is the entire killchain”

E2E Showcase - Generation



E2E Showcase - Output

```
Summary: 1/1 entities built · 0 skipped
  output: output/20260722T014156_build_a_machine_hosting_a_python_based_e

Artifacts written to: artifacts/2026-07-22T01-41-41
  LLM calls: 14  |  total tokens: 45,990  |  transcripts captured: 14
root@ip-172-31-19-142:/opt/GameOfEverything/game_of_everything#
```

The output directory contains the environment description, deployment script, solve script, and attacker procedure.

```
.
├── README.md
├── deploy.sh
├── playbook.yaml
└── solve.sh
```

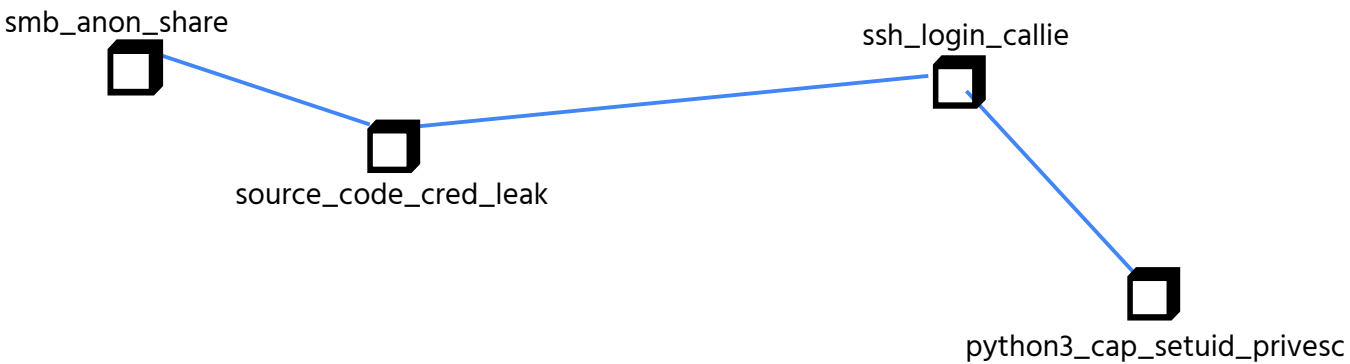


E2E Showcase - Deployed



Single-box Environment

“Build an a single server with a anonymous SMB share that hosts a zip file containing source code. A credential within that source code that is used to log onto a vulnerable machine via credential reuse to a non-admin user called 'callie'. Then use binary capabilities on python3 to pop a shell to escalate to root.”



Single-box Environment - Generation



Single-box Environment - Output

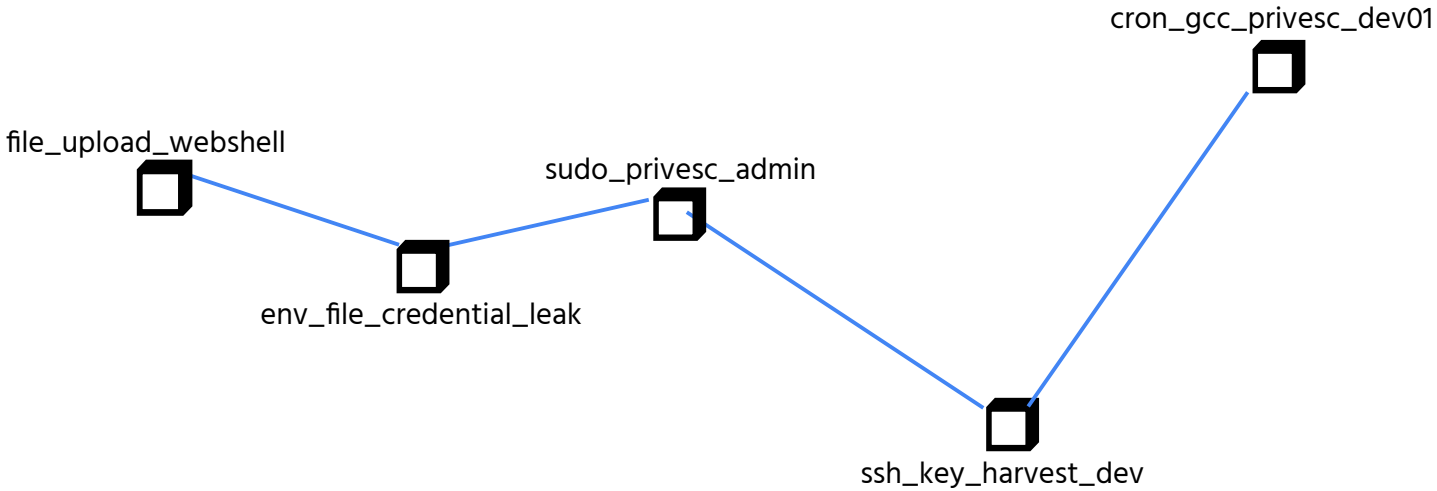
Multi-system environments include a docker-compose for container-based deployment and shell scripts for individual systems.

```
.
├── README.md
├── chain_playbook.yaml
├── deploy.sh
├── playbook.yaml
└── solve.sh
```



Multi-System Environment

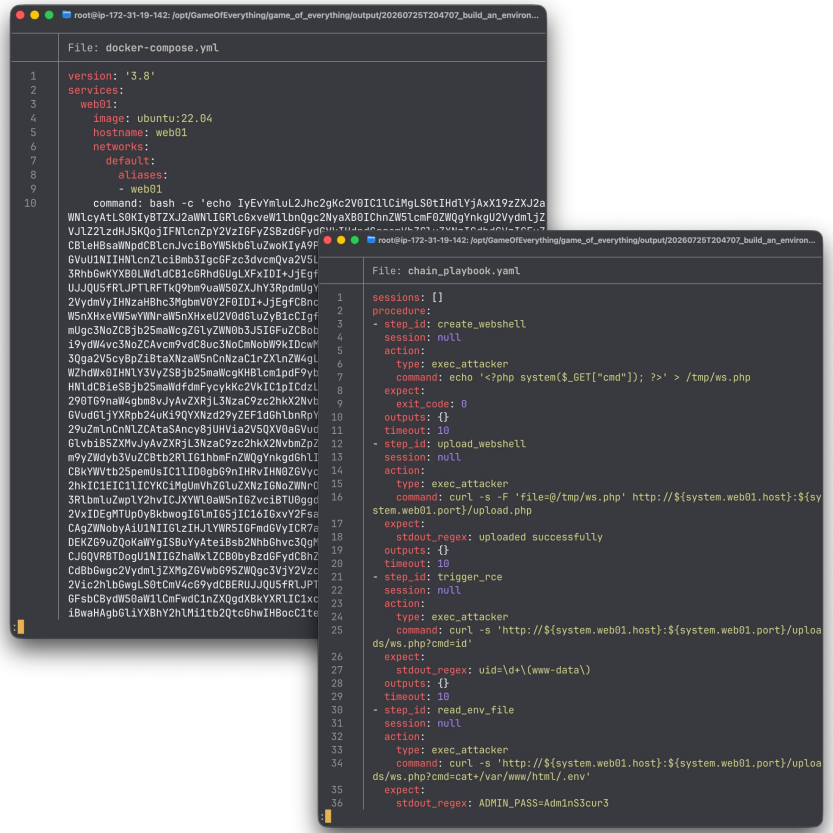
“Build an environment with two vulnerable servers. The first server (web01) contains a website with a PHP-based picture editing website that is vulnerable to arbitrary file upload. Via RCE on the first server, a .env file is exposed, containing credentials for user admin. Log in to admin via SSH and abuse sudo misconfigurations for root. Then discover an SSH key for user dev within root's home directory to SSH and pivot to the second server (dev01). On that second server, abuse a gcc building cronjob that is writable to escalate to root.”



Multi-System Environment - Output

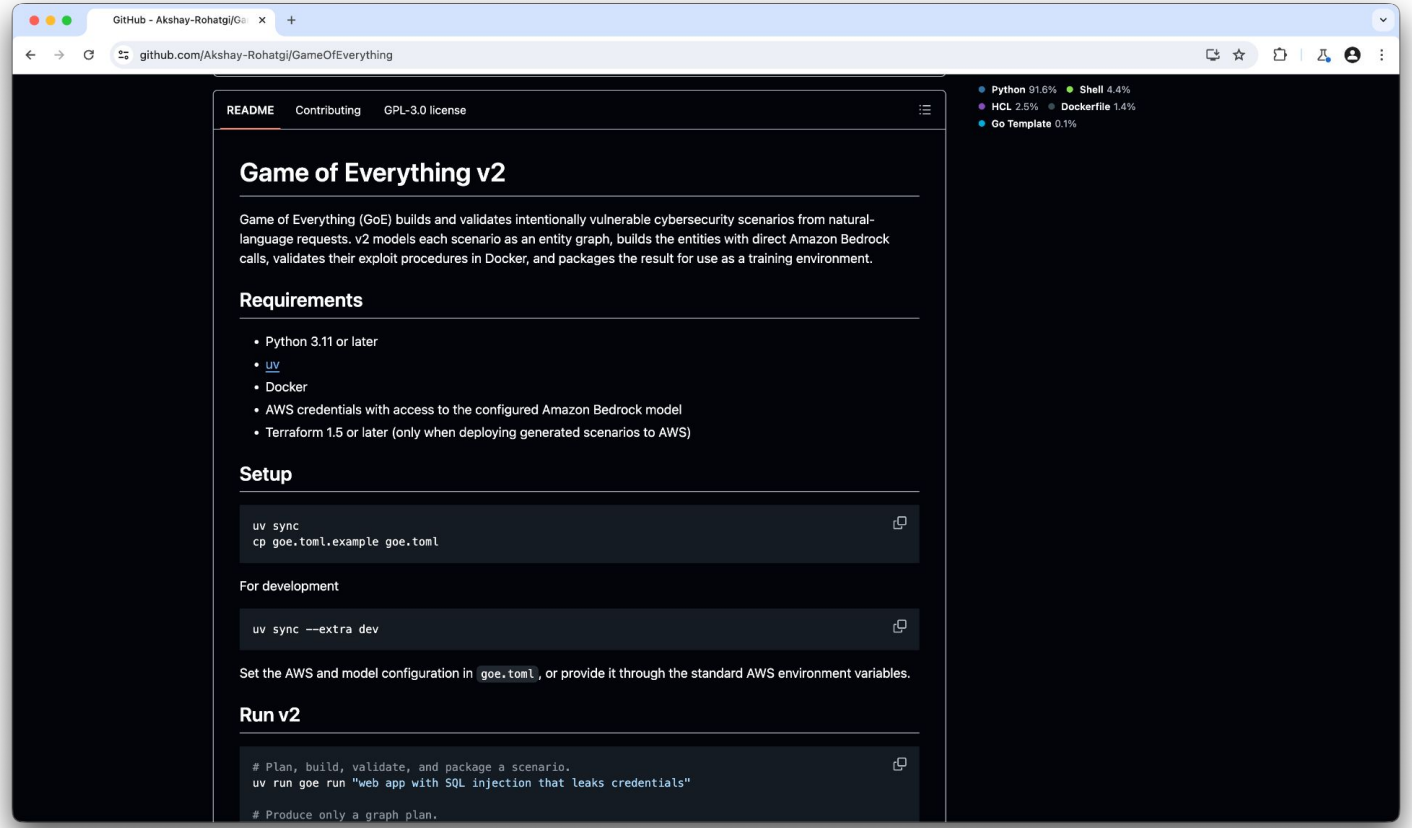
Multi-system environments include a docker-compose for container-based deployment and shell scripts for individual systems.

```
.
├── README.md
├── chain_playbook.yaml
├── dev01_deploy.sh
├── docker-compose.yml
├── playbook.yaml
├── solve.sh
└── web01_deploy.sh
```



What’s Next?

Open Source


GitHub - Akshay-Rohatgi/Ga
github.com/Akshay-Rohatgi/GameOfEverything
README
Contributing
GPL-3.0 license
Python 91.6%
Shell 4.4%
HCL 2.5%
Dockerfile 1.4%
Go Template 0.1%
Game of Everything v2
Game of Everything (GoE) builds and validates intentionally vulnerable cybersecurity scenarios from natural-language requests. v2 models each scenario as an entity graph, builds the entities with direct Amazon Bedrock calls, validates their exploit procedures in Docker, and packages the result for use as a training environment.
Requirements
Python 3.11 or later
uv
Docker
AWS credentials with access to the configured Amazon Bedrock model
Terraform 1.5 or later (only when deploying generated scenarios to AWS)
Setup
uv sync
cp goe.toml.example goe.toml
For development
uv sync --extra dev
Set the AWS and model configuration in goe.toml, or provide it through the standard AWS environment variables.
Run v2
Plan, build, validate, and package a scenario.
uv run goe run "web app with SQL injection that leaks credentials"
Produce only a graph plan.

Project Improvements

Varied Operating Systems

Next goal: Begin implementing runtimes other necessary changes to incorporate Debian-based and RHEL-based Linux systems into GoE.

Stretch goal: Figure out how to build harnesses to rapidly iterate upon and test Windows systems

Complex Applications

Next goal: Begin incorporating first-class, well-known web applications such as Wordpress or GitLab into GoE.

Stretch goal: Implement more custom runtimes and application types (e.g., APIs, TCP-based services)

Evaluation Framework

Next goal: Start logging the best examples that GoE provides to save them for future benchmark testing and regression tests.

https://github.com/Akshay-Rohatgi/GameOfEverything/blob/main/CONTRIBUTING.md

The Goals

1. Make it easier and faster to learn cyber
2. Training grounds for agentic evaluation
3. Community contribution

Questions?

https://github.com/Akshay-Rohatgi/GameOfEverything